

Pandas统计分析基础



- ◆ 了解pandas库
- ◆ 掌握Series对象的建立、常见属性和方法
- ◆ 掌握DataFrame对象的建立、常见属性和方法

- **Pandas (Python Data Analysis Library)** 是基于 **NumPy** 的数据分析库，提供了高性能易用的数据类型，以及大量能使我们快速便捷地处理数据的函数和方法，可以说 **Pandas** 是使得 **Python** 能够成为高效且强大的数据分析环境的重要因素之一。
- **Pandas** 有三种数据结构：**Series**、**DataFrame** 和 **Panel**。**Series** 类似于 一维数组；**DataFrame** 是类似表格的二维数组；**Panel** 可以视为 **Excel** 的多表单 **Sheet**。
- 导入方式：`import pandas as pd`

❖ Series

- **Series对象是一维数组结构，与NumPy中的一维数组ndarray类似，二者与Python基本的数据结构list也很相近，其区别是list中的元素可以是不同的数据类型，而一维数组ndarray和Series中则只允许存储同一数据类型的数据。**

❖ Series

- **Series** 是一种一维数组对象，包含了一个值序列，并且包含了数据标签，称为索引（**index**），可通过索引来访问数组中的数据。

Series	
index	values
0	'a'
1	'b'
2	'c'
3	'd'

1. Series对象创建

创建一个Series对象的最基语法格式如下：

pandas.Series(data=None,index=None,dtype=None, name=None)

返回值：返回一个Series对象

data：可以是一个Python列表，**index**与列表元素个数一致；也可以是字典，将键值对中的“值”作为Series对象的数据，将“键”作为索引；也可是一个标量值，这种情况下必须设置索引，标量值会重复来匹配索引的长度。

index：为Series对象的每个数据指定索引。

dtype：为Series对象的数据指定数据类型。

name：为Series对象起个名字。

1. Series对象创建

【例】用一维ndarray数组创建Series对象

```
import numpy as np
import pandas as pd
s=pd.Series(data=np.arange(0,5,2))
print(s)
```

```
0    0
1    2
2    4
dtype: int32
```

1. Series对象创建

【例】用标量值创建Series对象

```
import pandas as pd  
s2 = pd.Series(25, index = ['a', 'b', 'c'])  
print(s2)
```

```
a    25  
b    25  
c    25  
dtype: int64
```

1. Series对象创建

【例】通过列表创建Series

```
import pandas as pd
#仅有一个数组构成
obj = pd.Series([1, -2, 3, -4])
print(obj)
```

```
0    1
1   -2
2    3
3   -4
dtype: int64
```

1. Series对象创建

【例】通过指定索引的方式创建Series

```
i = ["a", "c", "d", "a"]  
v = [2, 4, 5, 7]  
t = pd.Series(v, index = i, name = "col")  
print(t)
```

```
a    2  
c    4  
d    5  
a    7  
Name: col, dtype: int64
```

尽管创建Series指定了index参数，实际Pandas还是有隐藏的index位置信息的。所以Series有两套描述某条数据的手段：位置和标签。

1. Series对象创建

【例4-3】Series位置和标签的使用

```
val=[2,4,5,6]
idx1=range(10,14)
idx2='hello the cruel world'.split()
s0=pd.Series(val)
s1=pd.Series(val,index=idx1)
t=pd.Series(val,index=idx2)
print(s0.index)
print(s1.index)
print(t.index)
print(s0[0])
print(s1[10])
print('default:',t[0],'label:',t['hello'])
```

```
RangeIndex(start=0, stop=4, step=1)
```

```
RangeIndex(start=10, stop=14, step=1)
```

```
Index(['hello', 'the', 'cruel', 'world'], dtype='object')
```

```
2
```

```
2
```

```
default: 2 label: 2
```

1. Series对象创建

2) 通过字典创建

如果数据被存放在一个Python字典中，也可以直接通过这个字典来创建Series。

```
sdata = {'Ohio': 35000, 'Texas': 71000, 'Oregon': 16000, 'Utah': 5000}  
obj3 = pd.Series(sdata)  
print(obj3)
```

```
Ohio      35000  
Texas     71000  
Oregon    16000  
Utah       5000  
dtype: int64
```

1. Series对象创建

2) 通过字典创建

【例】键值和指定的索引不匹配

```
sdata = {"a" : 100, "b" : 200, "e" : 300}  
letter = ["a", "b", "c", "e"]  
obj = pd.Series(sdata, index = letter)  
print(obj)
```

```
a    100.0  
b    200.0  
c      NaN  
e    300.0  
dtype: float64
```

1. Series对象创建

2) 通过字典创建

【例】不同索引数据的自动对齐

```
sdata = {'Ohio': 35000, 'Texas': 71000, 'Oregon': 16000, 'Utah': 5000}
obj1 = pd.Series(sdata)
states = ['California', 'Ohio', 'Oregon', 'Texas']
obj2 = pd.Series(sdata, index = states)
print(obj1+obj2)
```

```
California      NaN
Ohio            70000.0
Oregon          32000.0
Texas           142000.0
Utah            NaN
dtype: float64
```

1. Series对象创建

2) 通过字典创建

【例】Series索引的修改

```
obj = pd.Series([4, 7, -3, 2])  
obj.index = ['Bob', 'Steve', 'Jeff', 'Ryan']  
print(obj)
```

```
Bob      4  
Steve    7  
Jeff    -3  
Ryan     2  
dtype: int64
```

2. Series对象的属性

- (1) **shape**属性获取Series对象的形状。
- (2) **dtype**属性获取Series对象的数据数组中的数据的数据类型。
- (3) **values**属性获取Series对象的数据数组
- (4) **index**属性获取Series对象的数据数组的索引
- (5) Series对象本身及索引的**name**属性

2. Series对象的属性

```
In [4]: ▶ import pandas as pd
obj = pd.Series([4, 7, -3, 2])
obj.index = ['Bob', 'Steve', 'Jeff', 'Ryan']
print(obj)
print(obj.shape)
print(obj.dtype)
print(obj.values)
print(obj.index)
```

Bob 4
Steve 7
Jeff -3
Ryan 2
dtype: int64
(4,)
int64
[4 7 -3 2]
Index(['Bob', 'Steve', 'Jeff', 'Ryan'], dtype='object')

3. Series对象的数据的查看和修改

➤ 通过索引和切片查看Series对象的数据

- 可以使用数据索引以“Series对象[id]”方式访问Series对象的数据数组中索引为id的数据。

```
In [4]: ▶ s=pd.Series(data=[1,2,3], index = ['Java', 'C', 'Python'])
```

```
print(s['C'])
```

```
#可通过默认索引来读取
```

```
print(s[1])
```

```
#通过截取（切片）的方式读取多个元素
```

```
print(s[0:2])
```

```
2
```

```
2
```

```
Java    1
```

```
C        2
```

```
dtype: int64
```

3. Series对象的数据的查看和修改

➤ 通过索引和切片查看Series对象的数据

- 使用多个数据对应的索引来一次读取多个元素，注意索引要放在一个列表中。
- 根据筛选条件读取数据。

```
In [5]: ▶ s=pd.Series(data=[1,2,3], index = ['Java','C','Python'])  
print(s[['Python','C','Java']])  
print(s[s > 1])           #获取数据值>1的元素
```

```
Python    3  
C          2  
Java      1  
dtype: int64  
C          2  
Python    3  
dtype: int64
```

● Series对象的基本运算



1. Series对象的算术运算与函数运算

(1) 算术运算

适用于NumPy数组的运算符（+、-、*、/）或其它数学函数，也适用于Series对象。可以将Series对象的数据数组与标量进行+、-、*、/等算术运算。

(2) 函数运算

```
In [6]: ► s = pd.Series([2,4,6], index = ["a", "b", "c"])
          s1=np.sqrt(s)           #计算各数据的平方根
          s2=np.square(s)        #计算各数据的平方
          print(s1)
          print(s2)
```

```
a    1.414214
b    2.000000
c    2.449490
dtype: float64

a     4
b    16
c    36
dtype: int64
```

● Series对象的基本运算



2. Series对象之间的运算

➤ Series对象之间也可进行+、-、*、/等运算，不同Series对象运算的时候，能够**通过识别索引**进行**匹配计算**，即只有**索引相同的元素**才会进行相应的运算操作。

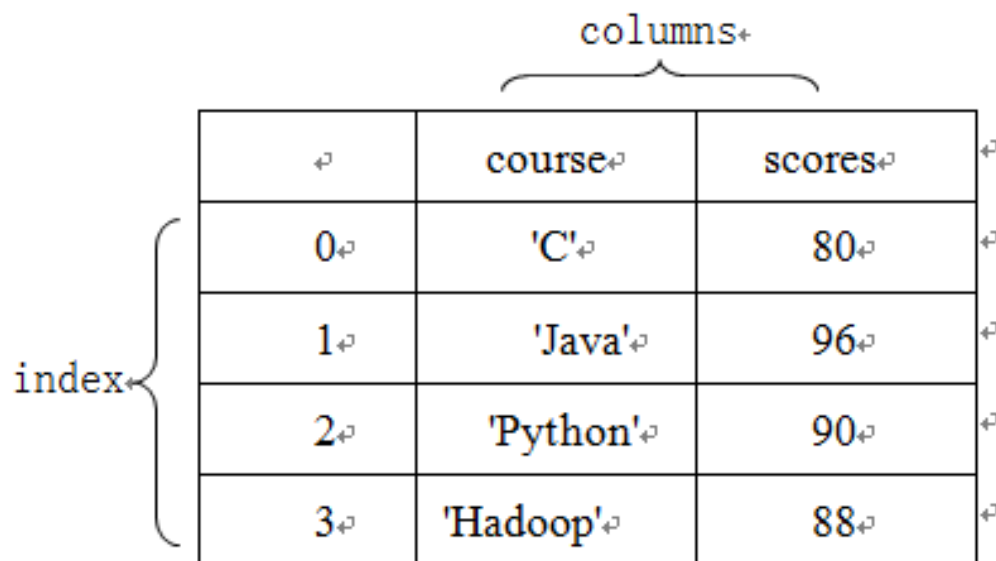
```
In [7]: ▶ s5=pd.Series([10,20],index=['c','d'])  
s6=pd.Series([2,4,6,8],index=['a','b','c','d'])  
print(s5+s6)                                #相同索引值的元素相加
```

```
a      NaN  
b      NaN  
c      16.0  
d      28.0  
dtype: float64
```

❖ DataFrame对象

DataFrame是一个**表格型**的数据结构，既有**行索引**（保存在**index**）又有**列索引**（保存在**columns**），是**Series**对象从一维到多维的扩展。**DataFrame**对象每列相同位置处的元素共用一个行索引，每行相同位置处的元素共用一个列索引。**DataFrame**对象各列的数据类型可以不相同。

DataFrame对象的内部组成如图所示



The diagram illustrates the internal structure of a DataFrame as a table. A bracket on the left labeled 'index' spans the first column. A bracket on top labeled 'columns' spans the subsequent two columns. Each cell in the table contains a small cursor icon (↵).

	course	scores
0	'C'	80
1	'Java'	96
2	'Python'	90
3	'Hadoop'	88

❖ DataFrame对象

● 创建DataFrame对象

创建DataFrame对象最常用的方法是使用pandas的DataFrame()构造函数，其语法格式如下：

Pd.DataFrame(data=None, index=None, columns=None, dtype=None)

返回值：DataFrame对象

参数说明：

data：创建DataFrame对象的数据，其类型可以是字典、嵌套列表、元组列表、numpy的ndarray对象、其它DataFrame对象。

index：行索引，创建DataFrame对象的数据时，如果没有提供索引，默认赋值为arange(n)。

columns：列索引，没有提供索引时，默认赋值为arange(n)。

dtype：用来指定元素的数据类型，如果为空，自动推断类型。

❖ DataFrame对象

● 创建DataFrame对象

(1) 可将一个字典对象传递给DataFrame()函数来生成一个DataFrame对象，字典的键作为DataFrame对象的列索引，字典的值作为列索引对应的列值，pandas也会自动为其添加一列从0开始的数值作为行索引。

```
data={'course':['C','Java','Python','Hadoop'], 'scores':[82,96,92,88], 'grade':['B','A','A','B']}  
df=pd.DataFrame(data)  
print(df)
```

	course	scores	grade
0	C	82	B
1	Java	96	A
2	Python	92	A
3	Hadoop	88	B

❖ DataFrame对象

● 创建DataFrame对象

(2) 可以只选择字典对象的一部分数据来创建DataFrame对象，只需在DataFrame构造函数中，用**columns选项**指定需要的列即可，新建的DataFrame对象各列顺序与指定的列顺序一致。

```
▶ data={'course':['C','Java','Python','Hadoop'], 'scores':[82,96,92,88], 'grade':['B','A','A','B']}  
df1=pd.DataFrame (data,columns=['course','grade'])  
print(df1)
```

	course	grade
0	C	B
1	Java	A
2	Python	A
3	Hadoop	B

❖ DataFrame对象

● 创建DataFrame对象

(3) 创建DataFrame对象时，如果没有用index数组明确指定行索引，pandas也会自动为其添加一列从0开始的数值作为行索引。如果想用自己定义的行索引，则要把定义的索引放到一个数组中，赋值给index选项。

```
data={'course':['C','Java','Python','Hadoop'], 'scores':[82,96,92,88], 'grade':['B','A','A','B']}  
df2=pd.DataFrame (data, index=['一','二','三','四'])  
print(df2)
```

	course	scores	grade
一	C	82	B
二	Java	96	A
三	Python	92	A
四	Hadoop	88	B

❖ DataFrame对象

● 创建DataFrame对象

(4) 创建DataFrame对象时，可以同时指定行索引和列索引，这时候就需要传递三个参数给DataFrame()构造函数，三个参数的顺序是：**数据**、**index选项**和**columns选项**。将存放行索引的数组赋给index选项，将存放列索引的数组赋给columns选项。

```
In [15]: ▶ df3=pd.DataFrame ([[1,2,3,4],[5,6,7,8],[9,10,11,12],[13,14,15,16]],  
                             index=['一','二','三','四'],columns=['A','B','C','D'])  
print(df3)
```

	A	B	C	D
一	1	2	3	4
二	5	6	7	8
三	9	10	11	12
四	13	14	15	16

❖ DataFrame对象

● 创建DataFrame对象

(5) 以字典的字典或Series的字典的结构创建DataFrame对象，pandas会将外边的键解释成列名称，将里面的键解释成行索引。

```
▶ data={ "name":{'one':"Jack",'two':"Mary",'three':"John",'four':"Alice"},  
         "age":{'one':10,'two':20,'three':30,'four':40},  
         "weight": {'one':30,'two':40,'three':50,'four':65}}  
df=pd.DataFrame(data)  
print(df)
```

	name	age	weight
one	Jack	10	30
two	Mary	20	40
three	John	30	50
four	Alice	40	65

❖ DataFrame对象

● 创建DataFrame对象

(6) 用键值为列表的字典构建DataFrame，其中每个列表（list）代表的是一个列，字典的键则是列索引。这里要注意的是每个列表中的元素数量应该相同，否则会报错。

```
In [17]: ▶ data1={ "name":["Jack","Mary","John"],  
                  "age": [10,20,30],  
                  "weight": [30,40,50]}  
df=pd.DataFrame(data1)  
print(df)
```

	name	age	weight
0	Jack	10	30
1	Mary	20	40
2	John	30	50

❖ DataFrame对象

● 创建DataFrame对象

(7) 以字典的列表构建DataFrame，其中每个字典代表的是每条记录（DataFrame中的一行），字典中各个键的键值对应的是这条记录的相关属性。

```
▶ d = [{'one' : 1, 'two' : 1}, {'one' : 2, 'two' : 2},  
       {'one' : 3, 'two' : 3}, {'two' : 4}]  
df = pd.DataFrame(d, index=['a', 'b', 'c', 'd'], columns=['one', 'two'])  
print(df)
```

	one	two
a	1.0	1
b	2.0	2
c	3.0	3
d	NaN	4

❖ DataFrame对象

● 创建DataFrame对象

(8) 以numpy对象构建DataFrame。

```
df4 = pd.DataFrame(np.arange(9).reshape(3, 3),  
index = ['a', 'c', 'd'], columns = ['one', 'two', 'four'])  
print(df4)
```

	one	two	four
a	0	1	2
c	3	4	5
d	6	7	8

❖ DataFrame对象的属性

属性名	功能描述
T	行列转置
columns	查看列索引名，可得到各列的名称
dtypes	查看各列的数据类型
index	查看行索引名
shape	查看DataFrame对象的形状
size	返回DataFrame对象包含的元素个数，为行数、列数大小的乘积
values	获取存储在DataFrame对象中的数据，返回一个NumPy数组
index.name	行索引的名称
columns.name	列索引的名称
loc	通过行索引获取行数据
iloc	通过行号获取行数据

❖ DataFrame对象的常用属性

属性	返回值
values	元素
index	索引
columns	列名
dtypes	类型
size	元素个数
ndim	维度数
shape	数据形状（行列数目）

❖ 查看DataFrame的常用属性

DataFrame的基础属性有**values**、**index**、**columns**、**dtypes**、**ndim**和**shape**，分别可以获取**DataFrame**的元素、索引、列名、类型、维度和形状。

❖ 查看DataFrame的常用属性

```
data3={  "":["one","two","three","four","five"],
        "status":["good","good","well","well","wonderful"],
        "year":["2012","2013","2014","2015","2016"],
        "index":[1,2,3,4,5]}
```

```
df=pd.DataFrame(data3)
```

```
print(df)
```

```
print('信息表的所有值为: \n',df.values)
```

```
print('信息表的所有列为: \n',df.columns)
```

```
print('信息表的元素个数为: \n',df.size)
```

```
print('信息表的维度是: \n',df.ndim)
```

```
print('信息表的形状为: \n',df.shape)
```

		status	year	index
0	one	good	2012	1
1	two	good	2013	2
2	three	well	2014	3
3	four	well	2015	4
4	five	wonderful	2016	5

信息表的所有值为:

```
[[ 'one' 'good' '2012' 1]
 [ 'two' 'good' '2013' 2]
 [ 'three' 'well' '2014' 3]
 [ 'four' 'well' '2015' 4]
 [ 'five' 'wonderful' '2016' 5]]
```

信息表的所有列为:

```
Index(['', 'status', 'year', 'index'], dtype='object')
```

信息表的元素个数为:

```
20
```

信息表的维度是:

```
2
```

信息表的形状为:

```
(5, 4)
```

❖ 索引对象

Pandas的索引对象负责管理轴标签和其它元数据（比如轴名称等）。构建**Series**或 **DataFrame**时，所用到的**任何数组或其他序列的标签**都会被转换成一个**Index**。

❖ 索引对象

【例】显示DataFrame的索引和列。

```
df = pd.DataFrame(np.arange(9).reshape(3, 3),  
index = ['a', 'c', 'd'], columns = ['one', 'two', 'four'])  
print(df)  
print(df.index)  
print(df.columns)
```

	one	two	four
a	0	1	2
c	3	4	5
d	6	7	8

```
Index(['a', 'c', 'd'], dtype='object')
```

```
Index(['one', 'two', 'four'], dtype='object')
```

❖ 索引对象

【例】 DataFrame的Index。

```
df = pd.DataFrame(np.arange(9).reshape(3, 3),  
index = ['a', 'c', 'd'], columns = ['one', 'two', 'four'])  
print('name' in df.columns)  
print('a' in df.index)
```

False

True

❖ 索引对象

每个**索引**都有一些**方法**和**属性**，它们可用于设置逻辑并回答有关该索引所包含的数据的常见问题。**Index**的常用方法和属性见下表。

方法	说明
append	连接另一个 Index 对象，产生一个新的 Index
diff	计算差集，并得到一个 Index
intersection	计算交集
union	计算并集
isin	计算一个指示各值是否都包含在参数集合中的布尔型数组
delete	删除索引 <i>i</i> 处的元素，并得到新的 Index
drop	删除传入的值，并得到新的 Index
insert	将元素插入到索引 <i>i</i> 处，并得到新的 Index
is_monotonic	当各元素均大于等于前一个元素时，返回 True
is.unique	当 Index 没有重复值时，返回 True
unique	计算 Index 中唯一值的数组

➤ 重新索引

- 索引对象是无法修改的，因此，**重新索引**是指**对索引重新排序**而不是重新命名，如果某个索引值不存在的话，会引入缺失值。

```
obj=pd.Series([7.2, -4.3, 4.5, 3.6], index=['b', 'a', 'd', 'c'])  
print(obj)  
obj.reindex(['a', 'b', 'c', 'd', 'e'])
```

```
b    7.2  
a   -4.3  
d    4.5  
c    3.6  
dtype: float64
```

对于重建索引引入的缺失值，可以利用fill_value参数填充。

```
a   -4.3  
b    7.2  
c    3.6  
d    4.5  
e    NaN  
dtype: float64
```

➤ 重新索引

■ **reindex**函数参数表

参数	使用说明
index	用于索引的新序列
method	插值（填充）方式
fill_value	缺失值替换值
limit	最大填充量
level	在 Multiindex 的指定级别上匹配简单索引，否则选取其子集
copy	默认为 True ，无论如何都复制；如果为 False ，则新旧相等时就不复制

➤ 重新索引

- 【例】重建索引时填充缺失值。

```
obj=pd.Series([7.2,-4.3,4.5,3.6],index=['b','a','d','c'])  
print(obj)  
obj.reindex(['a','b','c','d','e'],fill_value=0)
```

```
b    7.2  
a   -4.3  
d    4.5  
c    3.6  
dtype: float64
```

```
a   -4.3  
b    7.2  
c    3.6  
d    4.5  
e    0.0  
dtype: float64
```

➤ 重新索引

对于**顺序数据**，比如时间序列，重新索引时可能需要进行插值或填值处理，利用参数`method`选项可以设置：

`method = 'ffill'或'pad'`，表示前向值填充

`method = 'bfill'或'backfill'`，表示后向值填充

➤ 重新索引

- 【例】缺失值的前向填充。

```
import numpy as np
obj1=pd.Series(['blue','red','black'],index=[0,2,4])
obj1.reindex(np.arange(6),method='ffill')
```

```
0    blue
1    blue
2     red
3     red
4   black
5   black
dtype: object
```

➤ 重新索引

- 【例】缺失值的后向填充。

```
import numpy as np
obj2=pd.Series(['blue', 'red', 'black'], index=[0, 2, 4])
obj2.reindex(np.arange(6), method='backfill')
```

```
0    blue
1     red
2     red
3   black
4   black
5     NaN
dtype: object
```

❖ 更换索引

- 如果不希望使用默认的行索引，则可以在创建的时候通过Index参数来设置。
- 在DataFrame数据中，如果希望将**列数据作为索引**，则可以通过**set_index**方法来实现。

❖ 更换索引

【例】重建索引。

```
▶ data2={"name":["Alice","Jack","John","Mary" ],  
         "age":[40,10,30,20],  
         "weight":[65,30,50,40]}  
df=pd.DataFrame(data2,index=["four","one","three","two"])  
df1=df.set_index('name')  
print(df)  
print(df1)
```

	name	age	weight
four	Alice	40	65
one	Jack	10	30
three	John	30	50
two	Mary	20	40

	age	weight
name		
Alice	40	65
Jack	10	30
John	30	50
Mary	20	40

➤ DataFrame数据的编辑

1. 添加数据行

在Pandas DataFrame的末尾添加一行可以使用`df.loc()`函数。

```
data.loc[index, col] = value
```

`loc[]`中需要加入的是插入地方dataframe的索引，默认是整数型。

➤ DataFrame数据的编辑

1) 添加一行数据

在Pandas DataFrame的末尾添加一行可以使用`df.loc()`函数。

```
import pandas as pd
data={ "name":{"one":"Jack","two":"Mary","three":"John","four":"Alice"},
        "age":{"one":10,'two':20, 'three':30,'four':40},
        "weight": {'one':30,'two':40,'three':50,'four':65}}
df=pd.DataFrame(data)
print(df)
df.loc['five'] = ['harli', 33, 55]
df
```

	name	age	weight
one	Jack	10	30
two	Mary	20	40
three	John	30	50
four	Alice	40	65
	name	age	weight
one	Jack	10	30
two	Mary	20	40
three	John	30	50
four	Alice	40	65
five	harli	33	55

➤ DataFrame数据的编辑

2) 添加多行数据

添加多行可以使用 `_append()`，将一个 **DataFrame** 的行附加到另一个 **DataFrame** 的末尾。请注意，两个 **DataFrame** 应该具有相同的列名，以便成功地将一个 **DataFrame** 的行附加到另一个 **DataFrame** 的末尾。

● DataFrame的数据编辑



➤ DataFrame数据的编辑

2) 添加多行数据

```
import pandas as pd
#create DataFrame
df = pd.DataFrame({'points': [10, 12],
                   'rebounds': [7, 7],
                   'assists': [11, 8]})

#view DataFrame
print(df)
#define second DataFrame
df2 = pd.DataFrame({'points': [21, 25],
                   'rebounds': [7, 7],
                   'assists': [11, 3]})
|#add new row to end of DataFrame
df = df._append(df2, ignore_index = True)
#view updated DataFrame
df
```

	points	rebounds	assists
0	10	7	11
1	12	7	8

	points	rebounds	assists
0	10	7	11
1	12	7	8
2	21	7	11
3	25	7	3

➤ DataFrame数据的编辑

2) 添加多行数据

在Pandas DataFrame的
末尾添加多行也可以使用
pd.concat()函数。

```
import pandas as pd
#create DataFrame
df = pd.DataFrame({'points': [10, 12],
                   'rebounds': [7, 7],
                   'assists': [11, 8]})

#view DataFrame
print(df)

#define second DataFrame
df2 = pd.DataFrame({'points': [21, 25],
                   'rebounds': [7, 7],
                   'assists': [11, 3]})

#add new row to end of DataFrame
df = pd.concat([df, df2], axis=0, ignore_index = True)
#view updated DataFrame
df
```

	points	rebounds	assists
0	10	7	11
1	12	7	8
	points	rebounds	assists
0	10	7	11
1	12	7	8
2	21	7	11
3	25	7	3

➤ DataFrame数据的编辑

1. 添加数据列

增加列时，只需为要增加的**列赋值**即可创建一个新的列。

```
import numpy as np
data2={"name":["Alice","Jack","John","Mary"],
       "age":[40,10,30,20],
       "weight":[65,30,50,40]}
df=pd.DataFrame(data2,index=["four","one","three","two"])
print('原数据: \n',df)
df['math']=85
df['paper']=[82,90,86,88]
print(df)
```

原数据:

	name	age	weight
four	Alice	40	65
one	Jack	10	30
three	John	30	50
two	Mary	20	40

	name	age	weight	math	paper
four	Alice	40	65	85	82
one	Jack	10	30	85	90
three	John	30	50	85	86
two	Mary	20	40	85	88

➤ DataFrame数据的编辑

2. 删除数据

删除数据直接用**drop**方法，通过**axis**参数确定是删除的是**行**还是**列**。默认数据删除不修改原数据，需要在原数据删除行列需要设置参数**inplace = True**。

➤ DataFrame数据的编辑

2. 删除数据的行

```
import numpy as np
data2={"name":["Alice", "Jack", "John", "Mary" ],
       "age": [40, 10, 30, 20],
       "weight": [65, 30, 50, 40]}
df=pd.DataFrame(data2, index=["four", "one", "three", "two"])
print('原数据: \n', df)
print(df.drop('two'))
```

原数据:

	name	age	weight
four	Alice	40	65
one	Jack	10	30
three	John	30	50
two	Mary	20	40

	name	age	weight
four	Alice	40	65
one	Jack	10	30
three	John	30	50

➤ DataFrame数据的编辑

2. 删除数据的列

```
import numpy as np
data2={"name":["Alice","Jack","John","Mary" ],
        "age":[40,10,30,20],
        "weight":[65,30,50,40]}
df=pd.DataFrame(data2,index=["four","one","three","two"])
print('原数据: \n',df)
df.drop('age',axis=1,inplace=True)
print(df)
```

原数据:

	name	age	weight
four	Alice	40	65
one	Jack	10	30
three	John	30	50
two	Mary	20	40

	name	weight
four	Alice	65
one	Jack	30
three	John	50
two	Mary	40

➤ DataFrame数据的编辑

3. 修改数据

修改一个数据时直接对选择的数据赋值即可。

需要注意的是，数据修改是直接对DataFrame数据修改，操作无法撤销，因此更改数据时要做好数据备份。

➤ DataFrame数据的编辑

3. 修改一个数据

```
data2={"name":["Alice","Jack","John","Mary"],
        "age":[40,10,30,20],
        "weight":[65,30,50,40]}
df=pd.DataFrame(data2)
print(df)
df['name'][0]='Grace'
print(df)
```

	name	age	weight
0	Alice	40	65
1	Jack	10	30
2	John	30	50
3	Mary	20	40

	name	age	weight
0	Grace	40	65
1	Jack	10	30
2	John	30	50
3	Mary	20	40

```
<ipython-input-21-4c221a14dcd1>:6: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-y
df['name'][0]='Grace'
```

❖ DataFrame对象

3. 修改数据

可以用DataFrame对象的name属性为DataFrame对象的列索引columns和行索引index指定别的名称，以便于识别。

```
In [28]: data1={ "name":["Jack","Mary","John","Alice"],  
                 "age":[10,20,30,40],  
                 "weight":[30,40,55,65]}  
df=pd.DataFrame(data1)  
df.index.name='id'  
df.columns.name='item'  
print(df)
```

item	name	age	weight
id			
0	Jack	10	30
1	Mary	20	40
2	John	30	55
3	Alice	40	65

❖ DataFrame对象

3. 修改数据

可以为DataFrame对象**添加新的列**，指定新列的名称，以及为新列赋值。

```
In [29]: ▶ data1={ "name":["Jack","Mary","John","Alice"],  
                  "age": [10,20,30,40],  
                  "weight": [30,40,55,65]}  
df=pd.DataFrame(data1)  
#添加新列, 并都赋值为10  
df['new']=10  
print(df)
```

	name	age	weight	new
0	Jack	10	30	10
1	Mary	20	40	10
2	John	30	55	10
3	Alice	40	65	10

➤ DataFrame数据的查询

在数据分析中，选取需要的数据进行分析处理是最基本操作。
在Pandas中需要通过索引完成数据的选取。

1. 选取列：

通过列索引或以属性的方式可以单独获取DataFrame的列数据，返回的数据类型为Series。

➤ DataFrame数据的查询

1. 选取列:

```
import numpy as np
data2={"name":["Alice","Jack","John","Mary"],
       "age":[40,10,30,20],
       "weight":[65,30,50,40]}
df=pd.DataFrame(data2,index=["four","one","three","two"])
print('原数据: \n',df)
w1=df['name']
print('选取1列数据: \n',w1)
w2=df[['name','age']]
print('选取2列数据: \n',w2)
```

原数据:

	name	age	weight
four	Alice	40	65
one	Jack	10	30
three	John	30	50
two	Mary	20	40

选取1列数据:

four	Alice
one	Jack
three	John
two	Mary

Name: name, dtype: object

选取2列数据:

	name	age
four	Alice	40
one	Jack	10
three	John	30
two	Mary	20

➤ DataFrame数据的查询

2. 选取行

通过切片形式可以选取一行或多行数据。

```
import numpy as np
data2={"name":["Alice","Jack","John","Mary"],
       "age":[40,10,30,20],
       "weight":[65,30,50,40]}
df=pd.DataFrame(data2,index=["four","one","three","two"])
print('原数据: \n',df)
print('显示前2行: \n',df[:2])
print('显示2-3两行: \n',df[1:3])
```

原数据:

	name	age	weight
four	Alice	40	65
one	Jack	10	30
three	John	30	50
two	Mary	20	40

显示前2行:

	name	age	weight
four	Alice	40	65
one	Jack	10	30

显示2-3两行:

	name	age	weight
one	Jack	10	30
three	John	30	50

❖ DataFrame数据的查询

3. 查看DataFrame对象中的一个元素

要想获取存储在DataFrame对象中的一个元素，需要依次指定元素所在的列名称、行名称（索引）。

```
In [23]: ▶ data1={ "name":["Jack","Mary","John","Alice"],  
                  "age": [10, 20, 30, 40],  
                  "weight": [30, 40, 55, 65]}  
df=pd.DataFrame(data1)  
print(df)  
print(df['age'][1])
```

	name	age	weight
0	Jack	10	30
1	Mary	20	40
2	John	30	55
3	Alice	40	65

20

➤ DataFrame数据的筛选

4. 布尔选择

可以对**DataFrame**中的数据进行布尔方式选择。

```
import numpy as np
data2={"name":["Alice", "Jack", "John", "Mary" ],
        "age":[40, 10, 30, 20],
        "weight":[50, 30, 50, 40]}
df=pd.DataFrame(data2, index=["four", "one", "three", "two"])
df[df['weight']==50]
```

	name	age	weight
four	Alice	40	50
three	John	30	50

❖ DataFrame对象

● 判断元素是否属于DataFrame对象

可通过 **DataFrame** 对象的方法 **isin()** 判断一组元素是否属于 **DataFrame** 对象。

```
▶ data2={ "age": [40, 10, 30, 20],  
          "name": ["Alice", "Jack", "John", "Mary" ],  
          "weight": [65, 30, 50, 40]}  
df=pd.DataFrame(data2, index=["four", "one", "three", "two"])  
print(df)  
print(df.isin(['Jack', 30]))
```

	age	name	weight
four	40	Alice	65
one	10	Jack	30
three	30	John	50
two	20	Mary	40

	age	name	weight
four	False	False	False
one	False	True	True
three	True	False	False
two	False	False	False

❖ 数据运算与排序

DataFrame对象的数据运算与排序方法如表所示，表中的df表示一个DataFrame对象。

数据运算与排序方法	描述
<code>df.T</code>	df的行列转置
<code>df*N</code>	df的所有元素乘以N
<code>df1+df2</code>	将df1和df2的行名和列名都相同的元素相加，其它位置的元素用NaN填充
<code>df1.add(other, axis='columns', level=None, fill_value=None)</code>	将df1中的元素与other中的元素相加，other的类型可以是scalar（标量）、sequence（序列）、Series、DataFrame等形式
<code>df1.sub(other, axis='columns', level=None, fill_value=None)</code>	将df1中的元素与other中的元素相减
<code>df1.div(other, axis='columns', level=None, fill_value=None)</code>	将df1中的元素与other中的元素相除

❖ 数据运算与排序

DataFrame对象的数据运算与排序方法如表所示，表中的df表示一个DataFrame对象。

数据运算与排序方法	描述
df1.multiply(other, axis='columns', level=None, fill_value=None)	将df1中的元素与other中的元素相乘
df.apply(func,axis=0)	将func函数应用到df的行或列所构成的一维数组上
df.applymap(func)	将func函数应用到各个元素上
df.sort_index(ais=0, ascending=True)	按行索引进行升序排序
df.sort_values(by, axis=0, ascending=True)	按指定的列或行进行值排序
df.rank(axis=0, method='average', ascending=True)	沿着行计算元素值的排名，对于相同的两个元素值，沿着行顺序排在前面的数据排名高，返回各个位置上元素值从小到大排序对应的序号

❖ DataFrame数据的筛选

筛选DataFrame对象中的元素

可以通过指定条件筛选DataFrame对象的元素。

```
In [27]: data1={ "name":["Jack","Mary","John","Alice"],  
                 "age":[10,20,30,40],  
                 "weight":[30,40,55,65]}  
df=pd.DataFrame(data1)  
print(df[df.weight>35])  
print(df[df.age>35])
```

	name	age	weight
1	Mary	20	40
2	John	30	55
3	Alice	40	65

	name	age	weight
3	Alice	40	65

❖ DataFrame数据筛选

DataFrame对象的常用数据筛选方法如表所示

DataFrame对象的数据筛选方法	描述
<code>df.head(N)</code>	返回前N行
<code>df.tail(M)</code>	返回后M行
<code>df[m:n]</code>	切片，选取m~n-1行
<code>df[df['列名'] > value]</code>	选取满足条件的行
<code>df.query('列名 > value')</code>	选取满足条件的行
<code>df.query('列名 == [v1,v2,...]')</code>	选取列名列表的值等于v1,v2,...的行
<code>df.iloc[:,col]</code>	选取colname列的所有行
<code>df.iloc[row,col]</code>	选取某一元素
<code>df['col']</code>	获取col列，返回Series

❖ DataFrame数据的筛选

筛选DataFrame对象中的元素

选取通过DataFrame提供的head和tail方法可以得到多行数据，但是用这两种方法得到的数据都是从开始或者末尾获取连续的数据，而利用sample可以随机抽取数据并显示。

head () #默认获取前5行

head (n) #获取前n行

tail () #默认获取后5行

Tail (n) #获取后n行

sample (n) #随机抽取n行显示

❖ DataFrame数据的筛选

选取行和列

- ✓ **DataFrame.loc**(行索引名称或条件, 列索引名称)
- ✓ **DataFrame.iloc**(行索引位置, 列索引位置)

❖ DataFrame数据的筛选

选取行和列

```
import numpy as np
data2={"name":["Alice", "Jack", "John", "Mary" ],
       "age": [40, 10, 30, 20],
       "weight": [65, 30, 50, 40]}
df=pd.DataFrame(data2, index=["four", "one", "three", "two"])
#显示name和age两列
print(df.loc[:, ['name', 'age']])
#显示three和two两行中的name和age两列
print(df.loc[['three', 'two'], ['name', 'age']])
```

	name	age
four	Alice	40
one	Jack	10
three	John	30
two	Mary	20

	name	age
three	John	30
two	Mary	20

➤ DataFrame数据的筛选（利用iloc选取行和列）

```
import numpy as np
data2={"name":["Alice","Jack","John","Mary"],
       "age":[40,10,30,20],
       "weight":[65,30,50,40]}
df=pd.DataFrame(data2,index=["four","one","three","two"])
#显示第两列
print(df.iloc[:,2])
#显示第1和第3行的第1和第2列
print(df.iloc[[1,3],[1,2]])
```

```
four      65
one       30
three     50
two       40
Name: weight, dtype: int64

   age  weight
one  10     30
two  20     40
```

❖ DataFrame数据的筛选

```
data3={  "": ["one", "two", "three", "four", "five"],
         "status": ["good", "good", "well", "well", "wonderful"],
         "year": ["2012", "2013", "2014", "2015", "2016"],
         "index": [1, 2, 3, 4, 5]}
df=pd.DataFrame(data3, index=[1, 2, 3, 4, 5])
print(df)
print(df.iloc[1:4, 2]) #获取'year'列的1~3行
```

		status	year	index
1	one	good	2012	1
2	two	good	2013	2
3	three	well	2014	3
4	four	well	2015	4
5	five	wonderful	2016	5

2	2013
3	2014
4	2015

Name: year, dtype: object

➤ 算术运算

Pandas的数据对象在进行算术运算时，如果有相同索引则进行算术运算，如果没有，则会自动进行数据对齐，但会引入缺失值。

```
obj1=pd.Series([5.1, -2.6, 7.8, 10],  
               index=['a', 'c', 'g', 'f'])  
print('obj1:\n', obj1)  
obj2=pd.Series([2.6, -2.8, 3.7, -1.9],  
               index=['a', 'b', 'g', 'h'])  
print('obj2:\n', obj2)  
print(obj1+obj2)
```

```
obj1:  
a      5.1  
c     -2.6  
g      7.8  
f     10.0  
dtype: float64  
obj2:  
a      2.6  
b     -2.8  
g      3.7  
h     -1.9  
dtype: float64  
a      7.7  
b      NaN  
c      NaN  
f      NaN  
g     11.5  
h      NaN  
dtype: float64
```

➤ 算术运算

【例4-36】 DataFrame类型的数据相加。

```
a=np.arange(6).reshape(2,3)
b=np.arange(4).reshape(2,2)
df1=pd.DataFrame(a,columns=['a','b','e'],
                  index=['A','C'])
print('df1:\n',df1)
df2=pd.DataFrame(b,columns=['a','b'],
                  index=['A','D'])
print('df2:\n',df2)
print('df1+df2:\n',df1+df2)
```

df1:

	a	b	e
A	0	1	2
C	3	4	5

df2:

	a	b
A	0	1
D	2	3

df1+df2:

	a	b	e
A	0.0	2.0	NaN
C	NaN	NaN	NaN
D	NaN	NaN	NaN

❖ 函数应用和映射

已定义好的函数可以通过以下三种方法应用到数据：

1. **map**函数：将函数套用到**Series**的每个元素中；
2. **apply**函数，将函数套用到**DataFrame**的行或列上，行与列通过**axis**参数设置；
3. **applymap**函数，将函数套用到**DataFrame**的每个元素上。

❖ 函数应用和映射

【例】将水果价格表中的“元”去掉。

```
data = {'fruit': ['apple', 'grape', 'banana'],  
        'price': ['30元', '43元', '28元']}  
df1 = pd.DataFrame(data)  
print(df1)  
def f(x):  
    return x.split('元')[0]  
df1['price'] = df1['price'].map(f)  
print('修改后的数据表:\n', df1)
```

```
fruit price  
0  apple  30元  
1  grape  43元  
2  banana 28元  
修改后的数据表:  
fruit price  
0  apple   30  
1  grape   43  
2  banana  28
```

❖ 函数应用和映射

df.apply(func,axis=0)

作用：将func函数应用到DataFrame对象df的行或列所构成的一维数组上。

func：应用到行或列上的函数。

axis：**axis=0**，对每一列应用func函数；**axis=1**，对每一行应用函数。

DataFrame对象的基本运算



❖ 函数应用和映射

df.apply(func,axis=0)

```
▶ df4=pd.DataFrame([[0,1,2,3],[4,5,6,7],[8,9,10,11],[12,15,14,15]],
                    index=['A','B','C','D'],columns=['a','b','c','d'])
#计算数组元素的取值间隔
def f(x):
    return x.max()-x.min()
print(df4)
print(df4.apply(f,axis=0) )
```

	a	b	c	d
A	0	1	2	3
B	4	5	6	7
C	8	9	10	11
D	12	15	14	15
a	12			
b	14			
c	12			
d	12			

dtype: int64

❖ 函数应用和映射

【例】`apply`函数的使用方法。

```
df2=pd.DataFrame(np.random.randn(3,3),  
columns=['a','b','c'],index=['app','win','mac'])  
print(df2)  
df2.apply(np.mean)
```

	a	b	c
app	-0.520334	0.323528	0.291213
win	-0.329107	-0.055830	0.224756
mac	0.804510	0.363514	1.019145

a	-0.014977
b	0.210404
c	0.511704

dtype: float64

❖ 函数应用和映射

【例】`applymap`函数的用法。

```
df2=pd.DataFrame(np.random.randn(3,3),  
columns=['a','b','c'],index=['app','win','mac'])  
print(df2)  
df2.applymap(lambda x: '%.3f' %x)
```

	a	b	c
app	-0.344989	-0.644100	0.327233
win	0.153828	1.685012	0.060349
mac	0.896416	0.570038	-0.178651

	a	b	c
app	-0.345	-0.644	0.327
win	0.154	1.685	0.060
mac	0.896	0.570	-0.179

❖ 数据排序

df.sort_values(by, axis=0, ascending=True)

作用：按指定的列或行进行值排序。

参数说明：

by：指定某些行或列作为排序的依据。

axis：**axis=0**，对列进行排序；**axis=1**，对行进行排序。

❖ 数据排序

df.sort_values(by, axis=0, ascending=True)

```
▶ data5={ "col1":["A","A","B","D","C"],  
          "col2":[2,9,8,7,4],  
          "col3":[1,9,4,2,3]}  
df=pd.DataFrame(data5, index=[0,1,2,3,4])  
print(df)  
print(df.sort_values(by=['col1','col2']))
```

	col1	col2	col3
0	A	2	1
1	A	9	9
2	B	8	4
3	D	7	2
4	C	4	3

	col1	col2	col3
0	A	2	1
1	A	9	9
2	B	8	4
4	C	4	3
3	D	7	2

❖ 排序

sort_index方法：对索引进行排序，默认为升序，降序排序时加参数 **ascending=False**。

```
wy=pd.Series([1,-2,4,-4],index=['c','b','a','d'])  
print(wy)  
print('排序后的Series:\n',wy.sort_index())
```

```
c    1  
b   -2  
a    4  
d   -4  
dtype: int64  
排序后的Series:  
a    4  
b   -2  
c    1  
d   -4  
dtype: int64
```

❖ 排序

对于DataFrame数据排序，通过指定轴方向，使用`sort_index`函数对行或列索引进行排序。如果要进行列排序，则通过`sort_values`函数把列名传给`by`参数即可。

```
df2=pd.DataFrame(np.random.randn(3,3),
columns=['a','b','c'],index=['app','win','mac'])
print(df2)
print('排序后的DataFrame:\n',df2.sort_values('c'))
```

	a	b	c
app	1.423104	0.065977	0.688172
win	-1.220934	-0.091283	-0.074032
mac	1.775867	0.540152	1.199585

排序后的DataFrame:

	a	b	c
win	-1.220934	-0.091283	-0.074032
app	1.423104	0.065977	0.688172
mac	1.775867	0.540152	1.199585

❖ 数学统计

DataFrame对象的常用数学统计方法如表所示

数学统计方法	描述
<code>df.count(axis=0,level=None)</code>	统计每列或每行非NaN的元素个数。
<code>df.describe(percentiles=None, include=None, exclude=None)</code>	生成描述性统计，总结数据集分布的中心趋势，分散和形状，不包括NaN值。
<code>df.max(axis=0)</code>	<code>axis=0</code> 表示返回每列的最大值， <code>axis=1</code> 表示返回每行的最大值
<code>df.min(axis=0)</code>	<code>axis=0</code> 表示返回每列的最小值， <code>axis=1</code> 表示返回每行的最小值
<code>df.sum(axis=None,skipna=None, level=None)</code>	返回指定轴上元素值的和
<code>df.mean(axis=None,skipna=None, level=None)</code>	返回指定轴上元素值的平均值

❖ 数学统计

DataFrame对象的常用数学统计方法如表所示

数学统计方法	描述
<code>df.median(axis=None,skipna=None, level=None)</code>	返回指定轴上元素值的中位数
<code>df.var(axis=None,skipna=None, level=None)</code>	返回指定轴上元素值的均方差
<code>df.std(axis=None,skipna=None, level=None)</code>	返回指定轴上元素值的标准差
<code>df.cov()</code>	计算df的列列之间的协方差，不包括NA/空值
<code>df.corr(method='pearson')</code>	计算df的列列之间的相关性，不包括NA/空值
<code>df1.corrwith(df2)</code>	计算df1与df2的行或列之间的相关性
<code>df.cumsum(axis=0,skipna=True)</code>	对df求累加和，计算结果是与df形状相同的DataFrame对象
<code>df.cumprod(axis=None, skipna=True)</code>	返回df指定轴的元素累积积

❖ 汇总与统计

df.max(axis=0)

作用：**axis=0**表示返回df列的最大值，**axis=1**表示返回df行的最大值。

```
▶ df6=pd.DataFrame(np.arange(20).reshape(5,4),
                    index=list('abcde'),
                    columns=['one','two','three','four'])
print(df6)
print(df6.max(axis=1)) #返回每行的最大值
```

	one	two	three	four
a	0	1	2	3
b	4	5	6	7
c	8	9	10	11
d	12	13	14	15
e	16	17	18	19

a	3
b	7
c	11
d	15
e	19

dtype: int32

❖ 汇总与统计

在DataFrame中，可以通过sum方法对每列进行求和汇总，与Excel中的sum函数类似。如果设置axis = 1指定轴方向，可以实现按行汇总。

```
import numpy as np
df6=pd.DataFrame(np.arange(20).reshape(5,4),
                  index=list('abcde'),
                  columns=['one','two','three','four'])
print(df6)
print('按列汇总: \n',df6.sum())
print('按行汇总: \n',df6.sum(axis=1))
```

	one	two	three	four
a	0	1	2	3
b	4	5	6	7
c	8	9	10	11
d	12	13	14	15
e	16	17	18	19

按列汇总:

one	40
two	45
three	50
four	55

dtype: int64

按行汇总:

a	6
b	22
c	38
d	54
e	70

dtype: int64

❖ 汇总与统计

数据描述与统计：**describe()**方法可以对每个数值型列进行描述性统计，形成统计报告。

```
import numpy as np
df6=pd.DataFrame(np.arange(20).reshape(5,4),
                  index=list('abcde'),
                  columns=['one','two','three','four'])
print(df6)
df6.describe()
```

	one	two	three	four
a	0	1	2	3
b	4	5	6	7
c	8	9	10	11
d	12	13	14	15
e	16	17	18	19

26]:

	one	two	three	four
count	5.000000	5.000000	5.000000	5.000000
mean	8.000000	9.000000	10.000000	11.000000
std	6.324555	6.324555	6.324555	6.324555
min	0.000000	1.000000	2.000000	3.000000
25%	4.000000	5.000000	6.000000	7.000000
50%	8.000000	9.000000	10.000000	11.000000
75%	12.000000	13.000000	14.000000	15.000000
max	16.000000	17.000000	18.000000	19.000000

Pandas中常用的描述性统计量表

方法名称	说明	方法名称	说明
min	最小值	max	最大值
mean	均值	ptp	极差
median	中位数	std	标准差
var	方差	cov	协方差
sem	标准误差	mode	众数
skew	样本偏度	kurt	样本峰度
quantile	四分位数	count	非空值数目
describe	描述统计	mad	平均绝对离差

唯一值和值计数

通过Series对象的**unique()**方法可以获取不重复的数据，而通过Series对象的**value_counts()**方法可以统计每个值出现的次数。

```
df=pd.DataFrame({'Va1':[1,3,4,3,4], 'Va2':[2,3,1,2,3],  
                 'Va3':[1,5,2,4,4]})  
print('原DataFrame:\n', df)  
print('Va1频数统计: \n', df['Va1'].value_counts())  
print('Va3去重: \n', df['Va3'].unique())
```

原DataFrame:

	Va1	Va2	Va3
0	1	2	1
1	3	3	5
2	4	1	2
3	3	2	4
4	4	3	4

Va1频数统计:

4	2
3	2
1	1

Name: Va1, dtype: int64

Va3去重:

[1 5 2 4]

唯一值和值计数

```
obj=pd.Series(['a','b','c','a','d','c'])  
print('去重: \n',obj.unique())  
print('频数统计: \n',obj.value_counts())
```

去重:

['a' 'b' 'c' 'd']

频数统计:

c 2

a 2

d 1

b 1

dtype: int64

❖ 数据分组与聚合

对数据集进行**分组**并对**各分组应用函数**是数据分析中的重要环节。在pandas中，**分组运算**主要通过**groupby**函数来完成，**聚合**操作主要通过**agg**函数来完成。

分组的基本过程是，首先**数据集按照**分组键（**key**）的方式**分成小的数据片**（**split**），然后**对每一个数据片**进行**操作**，最后**将结果再组合**起来形成**新的数据集**（**combine**）。

❖ 数据分组与聚合

groupby对数据进行数据分组运算的过程分为三个阶段：分组、用函数处理分组和分组结果合并。

(1) **分组**。按照键（**key**）或者分组变量将数据分组。

分组键可以有多种形式，且类型不必相同：

①列表或数组，其长度与待分组的轴一样。

②**DataFrame**对象的某个列名。

③字典或**Series**，给出待分组轴上的值与分组名之间的对应关系。

④函数，用于处理轴索引或索引中的各个标签。

❖ 数据分组与聚合

groupby对数据进行数据分组运算的过程分为三个阶段：分组、用函数处理分组和分组结果合并。

(2) 用函数处理。

对于每个分组应用指定的函数，这些函数可以是Python自带函数，可以是自定义的函数。

(3) 合并分组处理结果。

把每个分组的计算结果合并起来。

(1) 数据分组

df.groupby(by=None, axis=0, level=None, as_index=True, sort=True, group_keys=True, squeeze=False)

作用：通过指定列索引或行索引，对df的数据元素进行分组。

参数名称	参数说明
by	可以传入函数、字典、 Series 等，用于确定分组的依据
axis	接收 int ，表示操作的轴方向，默认为 0
level	接收 int 或索引名，代表标签所在级别，默认为 None
as_index	接收 boolean ，表示聚合后的标签是否以 DataFrame 索引输出
sort	接收 boolean ，表示对分组依据和分组标签排序，默认为 True
group_keys	接收 boolean ，表示是否显示分组标签的名称，默认为 True
squeeze	接收 boolean ，表示是否在允许情况下对返回数据降维，默认 False

(1) 数据分组

```
import pandas as pd
import numpy as np
df=pd.DataFrame({'key1':['a','a','b','b','a'],
                 'key2':['yes','no','yes','yes','no'],
                 'data1':np.random.randn(5),
                 'data2':np.random.randn(5)})
grouped=df['data1'].groupby(df['key1'])
print(grouped.size())
print(grouped.mean())
```

```
key1
a      3
b      2
Name: data1, dtype: int64
key1
a      0.601711
b      0.271846
Name: data1, dtype: float64
```

① 按列名分组

DataFrame数据的列索引名可以作为分组键，但需要注意的是用于分组的对象必须是**DataFrame**数据本身，否则搜索不到索引名称会报错。另外，对分组后还包含其它非数值型数据进行某些聚合运算时程序也会报错。

```
import numpy as np
import pandas as pd
df=pd.DataFrame({'key1':['a','a','b','b','a'],
                  'data1':np.random.randn(5),
                  'data2':np.random.randn(5)})
groupk1=df.groupby('key1').mean()
groupk1
```

	data1	data2
key1		
a	-0.402412	-0.189600
b	-0.229172	1.002668

● DataFrame对象的基本运算



① 按列分组

```
▶ df7=pd.DataFrame([[5,2,'a','one'],[5,4,'a','two'],[3,5,'b','one'],  
                    [4,3,'b','two'],[1,2,'a','one']],  
                    columns=['data1','data2','key1','key2'],  
                    index=[0,1,2,3,4])  
print(df7)  
group = df7.groupby('key1')           #按'key1'进行分组  
print(group.groups)                  #通过groups属性来查看分组情况
```

	data1	data2	key1	key2
0	5	2	a	one
1	5	4	a	two
2	3	5	b	one
3	4	3	b	two
4	1	2	a	one

{ 'a': [0, 1, 4], 'b': [2, 3] }

② 按列表或元组分组

分组键还可以是长度和**DataFrame**行数相同的列表或元组，相当于将**列表或元组**看做**DataFrame**的**一行**，然后将其分组。

```
import pandas as pd
import numpy as np
df=pd.DataFrame({'key1':['a','a','b','b','a'],
                  'key2':['yes','no','yes','yes','no'],
                  'data1':np.random.randn(5),
                  'data2':np.random.randn(5)})
wlist=['w','w','y','w','y']
print(df)
df.groupby(wlist).sum()
```

	key1	key2	data1	data2
0	a	yes	0.947314	1.269842
1	a	no	1.431941	0.012438
2	b	yes	-0.543983	0.679599
3	b	yes	-0.186721	-1.115040
4	a	no	-0.258834	-2.128070

	data1	data2
w	2.192534	0.16724
y	-0.802818	-1.44847

③ 按字典分组

如果原始的**DataFrame**中的分组信息很难确定或不存在，可以通过字典结构，定义分组信息。

```
import pandas as pd
import numpy as np
df=pd.DataFrame(np.random.normal(size=(6,5)),
                 index=['a','b','c','A','B','c'])
print('数据为:\n',df)
wdict={'a':'one','A':'one','b':'two','B':'two','c':'three'}
print('分组汇总后的结果为: \n',df.groupby(wdict).sum())
```

数据为:

	0	1	2	3	4
a	0.042696	0.225693	-0.107744	0.694956	1.003646
b	-1.577733	0.205943	-1.437650	0.883364	0.258126
c	0.730963	-0.530852	-0.354678	-1.536843	0.037188
A	0.841993	1.588814	0.904094	-0.325836	-0.066414
B	-0.109740	-0.012086	-0.283350	-0.400133	0.636871
c	-0.008126	1.317431	-1.797387	1.300318	1.516330

分组汇总后的结果为:

	0	1	2	3	4
one	0.884689	1.814507	0.796349	0.369120	0.937232
three	0.722836	0.786579	-2.152065	-0.236525	1.553519
two	-1.687473	0.193857	-1.721000	0.483231	0.894998

④ 按函数分组

函数作为分组键的原理类似于字典，通过映射关系进行分组，但是函数更加灵活。

```
import pandas as pd
import numpy as np
def judge(x):
    if x>=0:
        return 'a'
    else:
        return 'b'
df=pd.DataFrame(np.random.randn(4,4))
print('数据为:\n',df)
print('函数作用后的结果为: \n',
      df[3].groupby(df[3].map(judge)).sum())
```

数据为:

	0	1	2	3
0	-2.555008	1.123457	2.203111	-0.266664
1	-0.174190	0.938632	-0.344343	1.906204
2	-0.613413	0.257744	-1.135788	-0.654092
3	-0.823226	-0.799792	1.257348	0.051889

函数作用后的结果为:

```
3
a    1.958093
b   -0.920755
Name: 3, dtype: float64
```

❖ 数据分组与聚合

(2) 按分组统计

在`df.groupby()`所生成的分组上应用`size()`、`sum()`、`count()`、`mean()`等统计函数，能分别统计分组数量、不同列的分组和、不同列的分组数量、分组不同列的平均值。

❖ 数据分组与聚合

(2) 按分组统计

```
▶ df7=pd.DataFrame([[5,2,'a','one'],[5,4,'a','two'],[3,5,'b','one'],  
                    [4,3,'b','two'],[1,2,'a','one']],  
                    columns=['data1','data2','key1','key2'],  
                    index=[0,1,2,3,4])  
  
print(df7)  
group = df7.groupby('key1')      #按'key1'进行分组  
print(group.size())              #统计分组数量  
print(group.sum())               #求不同列的分组和
```

```
   data1  data2 key1 key2  
0      5      2   a  one  
1      5      4   a  two  
2      3      5   b  one  
3      4      3   b  two  
4      1      2   a  one  
key1  
a      3  
b      2  
dtype: int64  
      data1  data2  
key1  
a         11      8  
b          7      8
```

❖ 数据聚合

1. 聚合运算

除了之前示例中的**mean**函数外，常用的聚合运算还有**count**和**sum**等。

函数	使用说明
count	计数
sum	求和
mean	求平均值
median	求中位数
std、var	无偏标准差和方差
min、max	求最小值最大值
prod	求积
first、last	第一个和最后一个值

2. 使用**agg**方法聚合数据

agg、**aggregate**方法都支持对每个分组应用某个函数，包括**Python**内置函数或自定义函数。同时，这两个方法也能够直接对**DataFrame**进行函数应用操作。

在正常使用过程中，**agg**和**aggregate**函数对**DataFrame**对象操作的功能基本相同，因此只需掌握一个即可。

❖ 数据分组与聚合

2. 使用agg方法聚合数据

对于聚合，一般指的是从数组产生标量值的数据转换过程，常见的聚合运算都有相关的统计函数快速实现，当然也可以自定义聚合运算。聚合操作主要通过agg函数来完成，agg函数的语法格式如下：

DataFrame.agg(func, axis=0)

作用：通过func在指定的轴上进行聚合操作。

func：用来指定聚合操作的方式，其数据形式有函数、字符串、字典以及字符串或函数所构成的列表。

axis：**axis=0**表示在列上操作，**axis=1**表示在行上操作。

❖ 数据分组与聚合

2. 使用agg方法聚合数据

(1) 在DataFrame对象的行或列上执行聚合操作

```
▶ data={'A':[1,2,7], 'B':[2,5,8], 'C':[3,6,9]}  
df8=pd.DataFrame(data)  
print(df8)  
print(df8.agg(['sum', 'min']))    #在df的各列上执行'sum'和'min'聚合操作
```

	A	B	C
0	1	2	3
1	2	5	6
2	7	8	9

	A	B	C
sum	10	15	18
min	1	2	3

❖ 数据分组与聚合

(3) 数据聚合

在`df.groupby()`所生成的分组上应用`agg()`

对于分组的某一系列或者多个列，应用`agg(func)`可以对分组后的数据应用`func`函数。

❖ 数据分组与聚合

(3) 数据聚合

```
data={'data1':[1,6,8,7,7,2,8,7], 'data2':[7,2,3,3,4,4,5,4],  
      'key1':['a','b','c','d','a','b','c','d'],  
      'key2':['one','two','three','one','two','three','one','two']}  
df9=pd.DataFrame(data)  
group9=df9.groupby('key1')  
print(df9)  
print(group9.agg('mean'))    #在df的各列上执行'mean'聚合操作
```

	data1	data2	key1	key2
0	1	7	a	one
1	6	2	b	two
2	8	3	c	three
3	7	3	d	one
4	7	4	a	two
5	2	4	b	three
6	8	5	c	one
7	7	4	d	two

	data1	data2
key1		
a	4.0	5.5
b	4.0	3.0
c	8.0	4.0
d	7.0	3.5

❖ 数据分组与聚合

(3) 数据聚合

在`df.groupby()`所生成的分组上应用`agg()`

```
data={'data1':[1,6,8,7,7,2,8,7], 'data2':[7,2,3,3,4,4,5,4],  
      'key1':['a','b','c','d','a','b','c','d'],  
      'key2':['one','two','three','one','two','three','one','two']}  
df9=pd.DataFrame(data)  
group9=df9.groupby('key2')  
print(df9)  
print(df9.groupby('key2').agg(['mean','sum']))#在每列上使用两个函数
```

	data1	data2	key1	key2
0	1	7	a	one
1	6	2	b	two
2	8	3	c	three
3	7	3	d	one
4	7	4	a	two
5	2	4	b	three
6	8	5	c	one
7	7	4	d	two

	data1		data2	
	mean	sum	mean	sum
key2				
one	5.333333	16	5.000000	15
three	5.000000	10	3.500000	7
two	6.666667	20	3.333333	10

❖ 数据分组与聚合

(3) 数据聚合

应用apply()函数执行聚合操作

```
▶ data={'data1':[1,6,8,7,7,2,8,7], 'data2':[7,2,3,3,4,4,5,4],  
        'key1':['a','b','c','d','a','b','c','d'],  
        'key2':['one','two','three','one','two','three','one','two']}  
df9=pd.DataFrame(data)  
print(df9)  
print(df9.groupby('key2').apply(sum))
```

	data1	data2	key1	key2
0	1	7	a	one
1	6	2	b	two
2	8	3	c	three
3	7	3	d	one
4	7	4	a	two
5	2	4	b	three
6	8	5	c	one
7	7	4	d	two

	data1	data2	key1	key2
key2				
one	16	15	adc	oneoneone
three	10	7	cb	threethree
two	20	10	bad	twotwotwo

❖ 分组运算

1. transform方法

通过transform方法可以将运算分布到**每一行**。

2. 使用apply方法聚合数据

apply方法类似于**agg**方法，能够将函数应用于**每一列**。

❖ 数据分组与聚合实例分析

(3) 数据分组与聚合

应用`agg()`函数执行聚合操作。

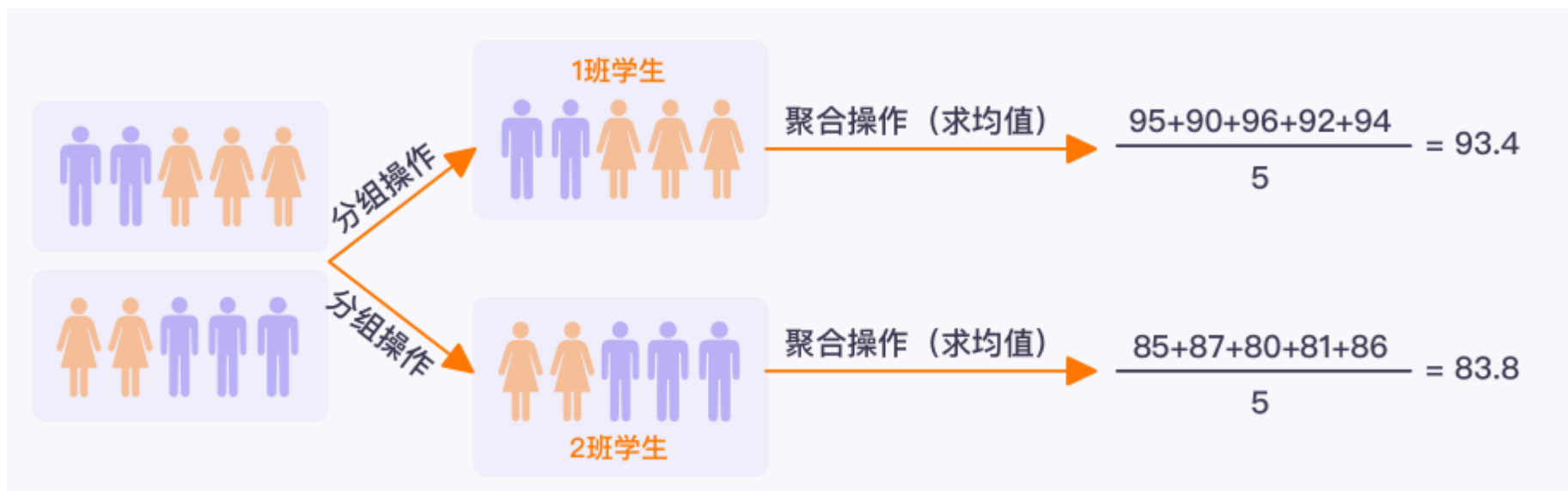
项目背景：王老师是某教育培训机构数学老师。现在他想研究一下他的学生数学期末考试成绩和什么因素有关。于是王老师先提取了一小部分学生的数据，保存在 **DataFrame** 对象中，王老师想要查看一下不同班级的学生数学平均分情况。就需要用到分组聚合操作。

❖ 数据分组与聚合实例分析

(3) 数据分组与聚合

分组聚合操作的定义：分组聚合操作指的是按照某项规则对数据进行分组，接着对分完组的数据执行总结性统计的操作（比如求和、求均值）。它可以帮助我们从此庞杂的数据中提炼出我们想要的的数据样式。

下图展现的就是利用分组聚合操作计算不同班级学生的均分的过程。



❖ 数据分组与聚合实例分析

(3) 数据分组与聚合

- 分组聚合操作根据其分组方式的不同可以分为**单层分组聚合**操作以及**多层分组聚合**操作。

- 单层分组聚合操作

单层分组聚合操作指的是针对某一个组进行聚合操作。王老师的案例就是单层分组聚合操作，因为他只需要针对**班级**这一列进行分组。

❖ 数据分组与聚合实例分析

(3) 数据分组与聚合

```
import pandas as pd
# 创建DataFrame对象
grade_df = pd.DataFrame({'班级': [1, 1, 1, 1, 1, 2, 2, 2, 2, 2],
                          '性别': ['男', '男', '女', '女', '女', '男', '男', '男', '女', '女'],
                          '眼镜': ['是', '否', '是', '否', '是', '是', '是', '否', '否', '否'],
                          '成绩': [95, 90, 96, 92, 94, 85, 87, 80, 81, 86]})
# 对班级进行分组聚合操作
grade_df1 = grade_df.groupby('班级')['成绩'].mean()
# 查看grade_df
print(grade_df)
# 查看grade_df1
print(grade_df1)
```

	班级	性别	眼镜	成绩
0	1	男	是	95
1	1	男	否	90
2	1	女	是	96
3	1	女	否	92
4	1	女	是	94
5	2	男	是	85
6	2	男	是	87
7	2	男	否	80
8	2	女	否	81
9	2	女	否	86

班级	成绩
1	93.4
2	83.8

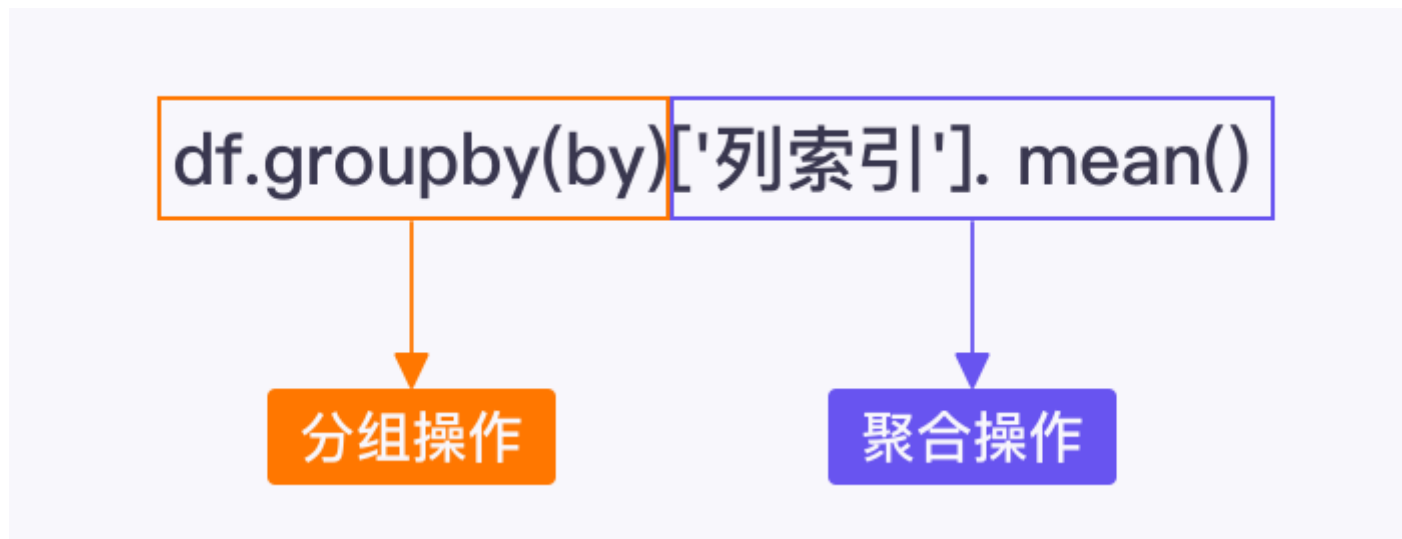
Name: 成绩, dtype: float64

❖ 数据分组与聚合实例分析

(3) 数据分组与聚合

● 单层分组聚合操作的代码：**df.groupby(by)['列索引'].mean()**。

这行代码可以拆分成两部分来看待，如下图所示：



❖ 数据分组与聚合实例分析

(3) 数据分组与聚合

● 首先来了解一下**单层分组操作**：`df.groupby(by)`。

参数 `by` 代表了要对哪一列数据进行分组操作。

这里要注意，单层分组操作只能根据一列数据进行分组。

如果要对班级进行分组，就可以将列索引'班级'传给参数 `by`。



❖ 数据分组与聚合实例分析

(3) 数据分组与聚合

● 代码效果如下图所示：

班级	性别	眼镜	成绩
1	男	是	95
1	男	否	90
1	女	是	96
1	女	否	92
1	女	是	94
2	男	是	85
2	男	是	87
2	男	否	80
2	女	否	81
2	女	否	86

对班级分组

班级	性别	眼镜	成绩
1	男	是	95
1	男	否	90
1	女	是	96
1	女	否	92
1	女	是	94
2	男	是	85
2	男	是	87
2	男	否	80
2	女	否	81
2	女	否	86

❖ 数据分组与聚合实例分析

(3) 数据分组与聚合

● 聚合操作

在分完组之后，就可以对某一系列数据进行聚合操作了。

`['列索引'].mean()` 中的'列索引'指的是需要对哪一系列数据进行聚合操作。如果想要对成绩这一列数据进行聚合操作，就可以将'成绩'传给'列索引'。

● DataFrame对象的基本运算

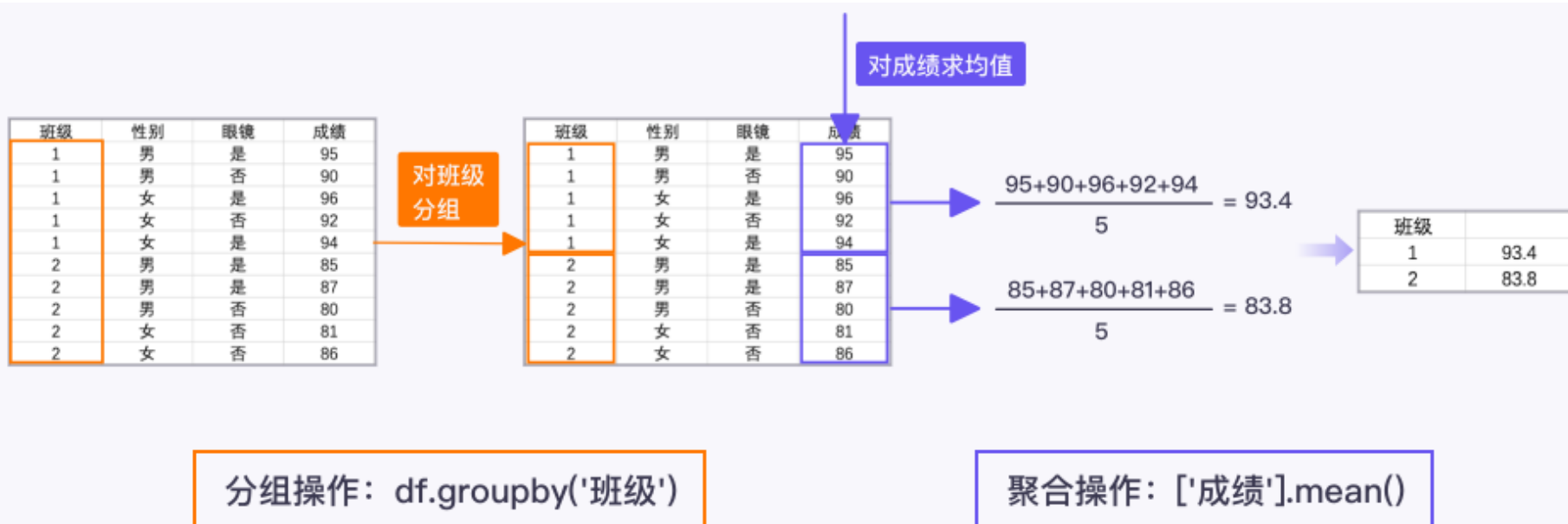


❖ 数据分组与聚合实例分析

(3) 数据分组与聚合

● 聚合操作

`mean()` 指的是求平均值。整体的效果如下图所示：



❖ 数据分组与聚合实例分析

(3) 数据分组与聚合

常用的聚合操作方法见如下表格。

流程	方法	示例
求最小值	min()	一组数据的最小值
求最大值	max()	一组数据的最大值
求标准差	std()	一组数据的标准差
求平均值	mean()	一组数据的总和除以一组数据的个数
求中位数	median()	一组数据中处于中间位置的数
求总和	sum()	一组数据的总和
求数量	count()	一组数据的非空数据的个数
求频数分布	value_counts()	一组数据各个类别出现的次数

❖ 数据分组与聚合实例分析

(3) 单层数据分组与聚合

● 聚合操作

帮助王老师求出不同性别学生的数学成绩中位数。

```
import pandas as pd
# 创建DataFrame对象
grade_df = pd.DataFrame({'班级': [1, 1, 1, 1, 1, 2, 2, 2, 2, 2],
                          '性别': ['男', '男', '女', '女', '女', '男', '男', '男', '女', '女'],
                          '眼镜': ['是', '否', '是', '否', '是', '是', '是', '否', '否', '否'],
                          '成绩': [95, 90, 96, 92, 94, 85, 87, 80, 81, 86]})

# 对班级进行分组聚合操作
grade_df2 = grade_df.groupby('性别')['成绩'].median()
# 查看grade_df1
print(grade_df2)
```

性别

女 92

男 87

Name: 成绩, dtype: int64

❖ 数据分组与聚合实例分析

(3) 单层数据分组与聚合

- 从运行结果中可以发现，分组聚合操作返回的是一个 **Series** 对象。
- 只不过这个 **Series** 对象和之前讲过的有些不一样，它的索引会多一个名字。
- 这个名字对应的是分组操作中的列索引。由于刚才是对性别这一列进行分组，所以名字就是性别。

之前

```
女    449
男    437
dtype: int64
```

现在

```
性别  女    449
      男    437
Name: 成绩, dtype: int64
```

索引的名字

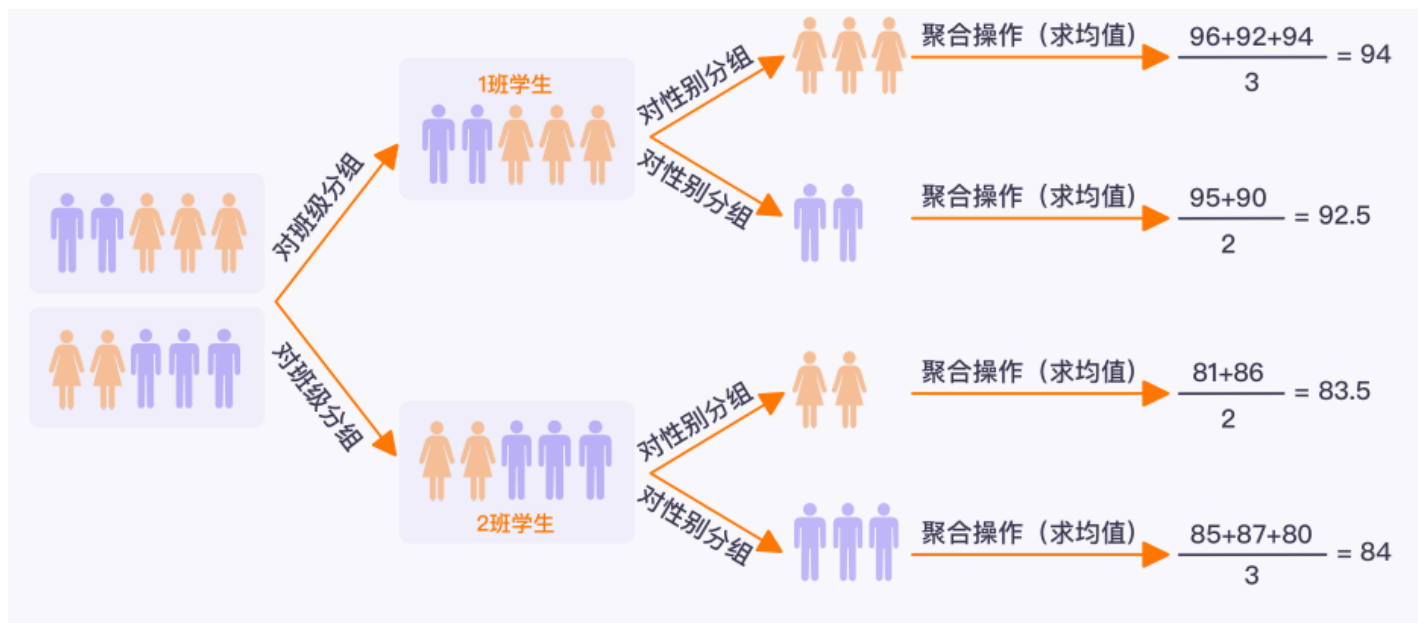
❖ 数据分组与聚合实例分析

(3) 多层分组与聚合

● 王老师还想对这组数据有进一步的研究，他想知道如何获取不同班级下不同性别的学生的平均分，这个时候就需要用到多层分组聚合操作的知识了。

● 多层分组聚合操作

下图展现的就是利用多层分组聚合操作计算不同班级下不同性别的学生均分的过程。



❖ 数据分组与聚合实例分析

(3) 多层数据分组与聚合

● 紧接着再运行下方的代码来体验一下。

```
❏ import pandas as pd
# 创建DataFrame对象
grade_df = pd.DataFrame({'班级': [1, 1, 1, 1, 1, 2, 2, 2, 2, 2],
                          '性别': ['男', '男', '女', '女', '女', '男', '男', '男', '女', '女'],
                          '眼镜': ['是', '否', '是', '否', '是', '是', '是', '否', '否', '否'],
                          '成绩': [95, 90, 96, 92, 94, 85, 87, 80, 81, 86]})
# 获取不同班级下不同性别的学生的平均分
grade_df3 = grade_df.groupby(['班级', '性别'])['成绩'].mean()
# 查看grade_df3
print(grade_df3)
```

班级 性别

1	女	94.0
	男	92.5
2	女	83.5
	男	84.0

Name: 成绩, dtype: float64

❖ 数据分组与聚合实例分析

(3) 多层数据分组与聚合

- 多层分组聚合和单层分组聚合相比，代码是相同的，都是 `df.groupby(by)['列索引'].mean()`。
- 从代码的运行结果来看，多层分组聚合操作返回的也是一个 **Series** 对象。
- 唯一的不同点在于索引的层数上。
- 单层分组聚合操作的索引只有一层，而多层分组聚合操作的索引至少有两层。

❖ 数据分组与聚合实例分析

(3) 多层数据分组与聚合

单层分组聚合操作

一层

班级

1 93.4

2 83.8

Name: 成绩, dtype: float64

多层分组聚合操作

二层

班级 性别

1 女 94.0

男 92.5

2 女 83.5

男 84.0

Name: 成绩, dtype: float64

❖ 数据分组与聚合实例分析

(3) 多层数据分组与聚合

- 层数的不同体现在参数 `by` 的赋值方式上。
- 在单层分组聚合操作的时候，只需要将一个列索引传给参数 `by` 就可以了。
- 而多层分组聚合操作需要传多个列索引给参数 `by`。
- 需要注意的是，这些列索引在传进参数 `by` 之前，需要先被放进一个“容器”里，这个容器可以是列表。

❖ 数据分组与聚合实例分析

(3) 多层数据分组与聚合



	班级	性别	
1	女		94.0
	男		92.5
2	女		83.5
	男		84.0
Name: 成绩, dtype: float64			

❖ 数据分组与聚合实例分析

(3) 多层数据分组与聚合

● 代码运行结果中索引的效果如下图所示：

● 仔细观察一下索引出现的次序。最左侧是班级，分为 1 班和 2 班。在 1 班和 2 班这两个类别下，又继续分为男和女这两个类别，也就是性别这列数据所对应的取值集合。



班级	性别
1	女
	男
2	女
	男

班级	性别	
1	女	94.0
	男	92.5
2	女	83.5
	男	84.0

Name: 成绩, dtype: float64

● DataFrame对象的基本运算

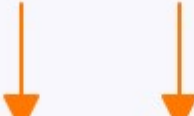


❖ 数据分组与聚合实例分析

(3) 多层数据分组与聚合

- 分组的顺序和列表中的参数是对应的（从左往右依次拆分）。

```
df.groupby(['班级', '性别'])['成绩'].mean()
```



	班级	性别	
1	女		94.0
	男		92.5
2	女		83.5
	男		84.0
Name: 成绩, dtype: float64			

❖ 数据分组与聚合实例分析

(3) 多层分组与聚合

- 如果现在想要先对性别分组，再对班级分组，那该如何向 **by** 传参呢？

```
import pandas as pd
# 创建DataFrame对象
grade_df = pd.DataFrame({'班级': [1, 1, 1, 1, 1, 2, 2, 2, 2, 2],
                          '性别': ['男', '男', '女', '女', '女', '男', '男', '男', '女', '女'],
                          '眼镜': ['是', '否', '是', '否', '是', '是', '是', '否', '否', '否'],
                          '成绩': [95, 90, 96, 92, 94, 85, 87, 80, 81, 86]})
# 获取不同班级下不同性别的学生的平均分
grade_df4 = grade_df.groupby(['性别', '班级'])['成绩'].mean()
# 查看grade_df3
print(grade_df4)
```

```
性别  班级
女    1    94.0
      2    83.5
男    1    92.5
      2    84.0
Name: 成绩, dtype: float64
```

❖ 数据分组与聚合实例分析

(3) 多层分组与聚合

● 假设现在王老师又想对数据有进一步的研究，他想知道尖子生是不是更容易出现在戴眼镜的学生群体中。于是王老师想要在原先分组的基础上，再对'眼镜'这一列进行拆分，

```
▶ import pandas as pd
# 创建DataFrame对象
grade_df = pd.DataFrame({'班级': [1, 1, 1, 1, 1, 2, 2, 2, 2, 2],
                          '性别': ['男', '男', '女', '女', '女', '男', '男', '男', '女', '女'],
                          '眼镜': ['是', '否', '是', '否', '是', '是', '是', '否', '否', '否'],
                          '成绩': [95, 90, 96, 92, 94, 85, 87, 80, 81, 86]})
# 获取不同班级、不同性别、戴与不戴眼镜的学生的最高分
grade_df5 = grade_df.groupby(['班级', '性别', '眼镜'])['成绩'].max()
# 查看grade_df5
print(grade_df5)
```

班级	性别	眼镜	
1	女	否	92
		是	96
	男	否	90
		是	95
2	女	否	86
	男	否	80
		是	87

Name: 成绩, dtype: int64

❖ 数据分组与聚合实例分析

(3) 多层分组与聚合

- 虽然通过多层分组聚合的方法，可以求出不同组别下的学生考试成绩的情况，但是返回的结果看起来却有些冗杂。
- 所有的组别信息都扎堆出现在了一个 **Series** 对象的索引中，有没有办法能够让这些信息看起来更直观一些呢？

班级	性别	眼镜
1	女	否 92.0
		是 95.0
	男	否 90.0
		是 95.0
2	女	否 83.5
	男	否 80.0
		是 86.0

Name: 成绩, dtype: float64

❖ 数据分组与聚合实例分析

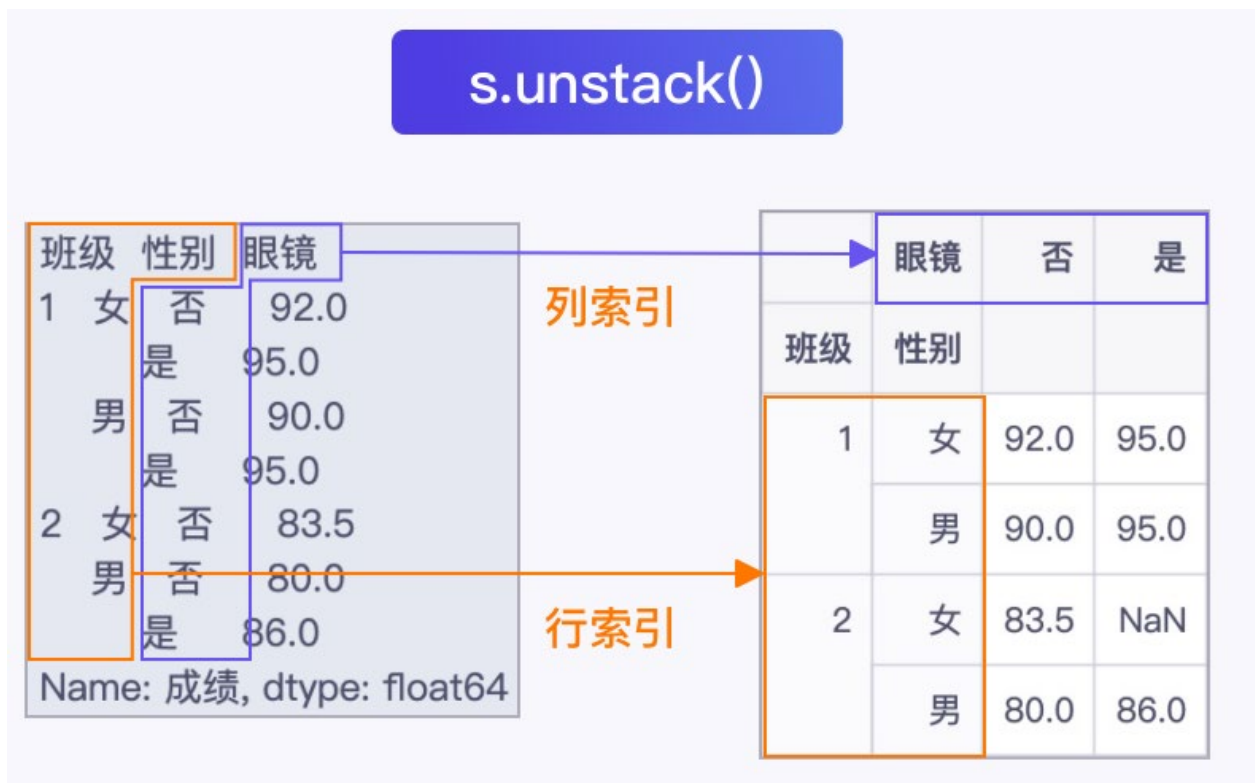
(3) 多层分组与聚合

- **s.unstack()** 函数就是解决这问题的常见方法。它可以将一个多层分组聚合后的 **Series** 对象转变成 **DataFrame** 对象。

❖ 数据分组与聚合实例分析

(3) 多层分组与聚合

● `s.unstack()` 这个方法是针对多层分组聚合后的 `Series` 对象来使用的。作用就是将其**索引的最后一列**转变成 `DataFrame` 对象的列索引，而剩下的索引则转变成 `DataFrame` 对象的行索引。



❖ 数据分组与聚合实例分析

(3) 多层分组与聚合

● 请通过下方代码来体验一下转变后的效果。

```
import pandas as pd
# 创建DataFrame对象
grade_df = pd.DataFrame({'班级': [1, 1, 1, 1, 1, 2, 2, 2, 2, 2],
                          '性别': ['男', '男', '女', '女', '女',
                                    '男', '男', '男', '女', '女'],
                          '眼镜': ['是', '否', '是', '否', '是',
                                    '是', '是', '否', '否', '否'],
                          '成绩': [95, 90, 96, 92, 94, 85, 87,
                                    80, 81, 86]})
# 获取不同班级、不同性别、戴与不戴眼镜的学生的最高分
grade_df5 = grade_df.groupby(['班级', '性别', '眼镜'])['成绩'].max()
df6 = grade_df5.unstack()
# 查看grade_df6
df6
```

		眼镜	否	是
班级	性别			
1	女	92.0	96.0	
	男	90.0	95.0	
2	女	86.0	NaN	
	男	80.0	87.0	

❖ 数据分组与聚合实例分析

(3) 多层分组与聚合

● 接下来就来做一道练习，要求是获取如下图所示的表格（提示：聚合操作的方法是求平均值）。

● 首先可以从表格中发现，列索引是'性别'，所以在给参数 `by` 赋值的时候，应该将'性别'放在列表的末尾。其次，从 `DataFrame` 对象的行索引中可以发现，'班级'在左，'眼镜'在右，所以最后传给参数 `by` 的列表应该是：['班级', '眼镜', '性别']。

	性别	女	男
班级	眼镜		
1	否	92.0	90.0
	是	95.0	95.0
2	否	83.5	80.0
	是	NaN	86.0

❖ 数据分组与聚合实例分析

(3) 多层分组与聚合

● 代码如下图。

```
import pandas as pd
# 创建DataFrame对象
grade_df = pd.DataFrame({'班级': [1, 1, 1, 1, 1, 2, 2, 2, 2, 2],
                          '性别': ['男', '男', '女', '女', '女', '男', '男', '男', '女', '女'],
                          '眼镜': ['是', '否', '是', '否', '是', '是', '是', '否', '否', '否'],
                          '成绩': [95, 90, 96, 92, 94, 85, 87, 80, 81, 86]})

# 获取上图所示的表格
grade_df7 = grade_df.groupby(['班级', '眼镜', '性别'])['成绩'].mean().unstack()
# 查看grade_df6
grade_df7
```

]:

		性别	
		女	男
班级	眼镜		
1	否	92.0	90.0
	是	95.0	95.0
2	否	83.5	80.0
	是	NaN	86.0